HeavyMatcher: Finding Top-k Items with Compact Linear Space

Han Wang*, Sheng Lan†*, Jiayao Wang‡, Qilong Shi§, Wenjun Li*✉,
Laizhong Cui¶, Tong Yang‖, Shuhui Chen‡, Weizhe Zhang‡‡*
*Pengcheng Laboratory, Shenzhen, China †Peking University Shenzhen Graduate School, Shenzhen, China
‡National University of Defense Technology, Changsha, China §Tsinghua University, Beijing, China
¶Shenzhen University, Shenzhen, China ‖Peking University, Beijing, China ‡‡Harbin Institute of Technology, Harbin, China

***Abstract*—Finding Top-k items is fundamental to large-scale data stream processing and network traffic measurement. Sketch algorithms, celebrated for their compact and probabilistic nature, are widely deployed in practice. However, existing methods often struggle to strike a satisfactory balance between accuracy and space efficiency, leading to significant trade-offs. In Top-k detection, to accurately identify the k most frequent items, the data structure must allocate at least k entries (i.e., <*item ID, frequency*> pairs). Achieving perfect (100%) precision often demands additional memory—typically allocating one or two orders of magnitude more than k entries on average to mitigate hash collisions—resulting in substantial but unnecessary memory overhead. To address this, we introduce HeavyMatcher, a novel approach that leverages entry merging and evolution techniques. By employing cuckoo hashing for better load balancing, we further compact memory usage, ensuring the Top-k items are precisely retained within the data structure. Our theoretical analysis and extensive experiment results demonstrate that HeavyMatcher outperforms other Top-k sketches in terms of error by up to four orders of magnitude, while also exhibiting superior precision and lower space requirement. Notably, the minimum required space of HeavyMatcher to attain perfect precision on various highly skewed datasets used in our experiment is $1.43 \times k$ entries on average, achieving near-optimal linear space for finding Top-k items. Furthermore, we deploy HeavyMatcher in a persistent network incident response system, reducing benign traffic damage by nearly 50% while substantially improving mitigation effectiveness. Open-source code and a supplementary technical report are available on GitHub[1].**

***Index Terms*—Data stream, Top-k, Perfect precision, Compact linear space, Sketch.**

I. Introduction

Finding Top-k items has gained considerable attention in data stream processing, with broad applications in data mining [1]–[7], database optimization [8]–[11], and network traffic measurement [12]–[18]. This process involves detecting items to report the k most frequent ones within an evolving data stream. In the era of big data, where the candidate set of hot items is massive, Top-k detection faces stringent constraints on both memory efficiency and accuracy [19]–[21]. Typical examples include network monitoring and persistent traffic mitigation under tight device memory, where compact, low-latency, and highly accurate Top-k detection is critical.

To detect Top-k items, traditional methods struggle to balance accuracy and efficiency under high-speed data streams, prompting a shift toward compact probabilistic approaches. Among them, sketches have emerged as a prominent data structure, recognized for their capability to summarize large data streams while preserving essential frequency information [22]–[30]. Their effectiveness in real-time processing and querying has gained widespread acceptance for Top-k tasks in data stream processing [31]–[36].

Sketch algorithms known for Top-k tasks commonly organize entries (i.e., <*item ID, frequency*> pairs) into clever data structures, to effectively preserve potential item information alongside their estimated frequencies. For instance, the classic Space-Saving (SS) [37] employs a Stream-Summary structure akin to a heap, enabling online queries in $\Theta(1)$ time. Additionally, the notable HeavyKeeper (HK) [38] enhances accuracy by using a structure resembling Count-Min Sketch (CMS) [39]. And recently, Double-Anonymous Sketch (DAS) [40] utilizes a bucket-connected structure that not only maintains high accuracy but also ensures Top-k-fairness.

Despite these advancements, the above entry-based methods still face inherent trade-offs between accuracy and space efficiency. For example, while pursuing perfect (100%) Top-k detection, it may seem intuitive that the optimal minimum memory usage is just k entries; however, the reality is that the state-of-the-art (SOTA) Top-k sketch algorithm, DAS, requires about an order of magnitude more entries than this ideal memory (k), while the classic SS and other algorithms demand even more, as shown by our analyses of a simulated skewed dataset in §II-C. This finding is universal, as most real-world data streams exhibit similar skewed distributions, consistent with the Pareto principle [41]. The trade-off is difficult to break because hash collisions can complicate estimation and will lead to potential competition among items, necessitating the allocation of substantially more memory. This prompts an intriguing question: ***Can we devise a sketch algorithm that achieves perfect Top-k detection while maintaining compact linear space, thereby breaking this trade-off in prevalent skewed data streams?*** Specifically, achieving near-optimal

Han Wang, Sheng Lan, and Jiayao Wang are co-first authors of this paper. This work was conducted at Pengcheng Laboratory under the supervision of the corresponding author, Wenjun Li (wenjunli@pku.org.cn).

[1]https://github.com/wenjunpaper/HeavyMatcher

979-8-3195-0662-7/26/$31.00 ©2026 IEEE

linear space in Top-k detection happens when the required number of entries increases linearly with k, with the growth ratio approaching 1.

To answer this question, we propose a novel Top-k sketch algorithm named **HeavyMatcher** (HM). It adaptively preserves as many **heavy** (hot) items as possible for accurate detection of Top-k items while optimizing memory utilization by real-time **matching** of limited memory with these items, to achieve perfect precision with compact linear space requirements. The primary design concept of HeavyMatcher involves dynamically adjusting the localized bucket structure as the data stream arrives. This is achieved through entry merging and evolution techniques that help prevent the incorrect discarding of hot items. Additionally, cuckoo hashing is employed for overall optimization of memory utilization, specifically to address extreme cases where multiple hot items might occupy the same localized bucket. By preventing such cases, cuckoo hashing mitigates the risk of discarding critical hot items due to memory constraints within these buckets. The design ideas of HeavyMatcher revolve around the following key points:

1) **Efficient Entry Organization in Buckets:** The proposed data structure features optimized buckets for enhanced Top-k estimations. Each bucket contains fixed-length ID entries and fingerprint (FP) entries. ID entries store hot items as complete <*item ID, frequency*> pairs, minimizing hash collisions and ensuring high precision. In contrast, FP entries hold cold items as shorter <*fingerprint, frequency*> pairs, efficiently utilizing limited memory to accommodate cold items and alleviating their competition for Top-k inclusion.
2) **Evolution with Entry Merging:** Buckets evolve through entry merging, with each bucket marked by a flag indicating its state. Items in a bucket are sorted in real-time. When the estimated frequency in an FP entry exceeds a dynamic threshold, that item is considered a potential Top-k candidate, triggering an evolution process that transitions the FP entry to an ID entry. This design protects hot items from competition with cold items in time, thereby enhancing the Top-k detection precision.
3) **Load Balancing Among Buckets:** Memory utilization is optimized through load balancing via cuckoo hashing. This method extends evolution from individual buckets to the entire structure, reducing the risk of rapid overflow in a bucket to ensure Top-k items precisely retained. During evolution, items in a bucket's ID entries are examined, and the item whose alternate bucket has the fewest ID entries (i.e., the lowest flag value) is kicked out to its alternate bucket. This can further triggers a chain reaction of continuous kicks, facilitating overall adjustment across buckets and compacting memory usage.
4) **Compact Linear Space Requirement:** We establish a theoretical proof that reveals a linear growth relationship between the minimum required entries and k for perfect Top-k detection in a dataset. To further validate this relationship, we executed theoretical simulations and experiments using different skewed datasets. The experimental results show that HeavyMatcher requires only $1.43 \times k$ entries on average in such high-skew scenarios, indicating that our sketch provides a near-optimal solution in terms of compact linear space for precisely finding Top-k items.

We implemented HeavyMatcher in C++ and conducted comprehensive performance evaluations across various datasets. The results are noteworthy. In real-world datasets, HeavyMatcher's error performance outperforms that of other Top-k sketch algorithms by up to four orders of magnitude. Furthermore, HeavyMatcher demonstrates significantly higher precision compared to existing algorithms, and its space-efficient design allows it to converge to 100% precision more rapidly. It also exhibits clear throughput advantages, particularly under constrained memory. Additionally, on the evaluated high-skew workloads, HeavyMatcher achieves near-optimal minimum space requirements for perfect Top-k detection. To further demonstrate its practical value, we deploy HeavyMatcher in a persistent network incident response system, showing its effectiveness in improving mitigation success while reducing benign traffic damage.

Roadmap: §II provides the background and related work of Top-k sketches. §III presents the detailed design of HeavyMatcher. §IV includes the mathematical analysis, while §V presents the experimental evaluation, including a practical application case study. Finally, §VI concludes this paper.

II. Background and Motivation

A. Problem Statement

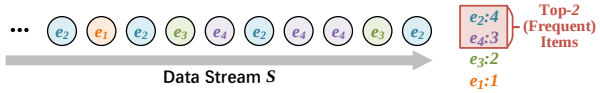


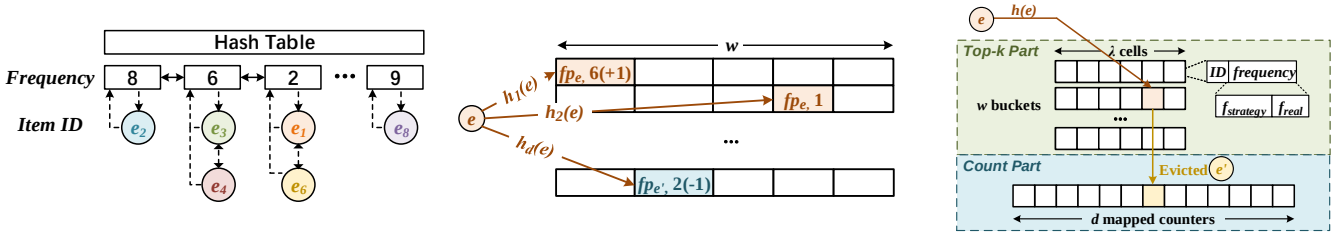
Fig. 1: Finding Top-k items.

Data Stream Model: As shown in Fig. 1, a data stream $S = \{s_1, s_2, \ldots, s_N\}$ is a sequence of N items processed in real time. These items can be categorized into n distinct, non-overlapping items represented by the set $E = \{e_1, e_2, \ldots, e_n\}$. Items in set E may occur multiple times within the stream, with frequencies f_i such that $\sum_{i=1}^{n} f_i = N$. For instance, in a network stream e_i might denote a flow while each s_j corresponds to an individual packet.

Top-*k* Items Detection: The Top-k items detection task focuses on finding the k frequent items within a data stream, providing both item IDs and their corresponding frequencies, represented as $\Phi = \{(e_1.id, f_1), (e_2.id, f_2), \ldots, (e_k.id, f_k)\}$. If the reported Top-$k$ item IDs are identical to the actual Top-k item IDs, we consider it perfect detection with 100% precision.

B. Related Work

For Top-k detection in data streams, various sketches have been extensively employed. Depending on how entries are organized, existing approaches can be broadly classified into three categories: heap-like, CMS-like, and bucket-connected solutions. Below, we briefly review each class.



(a) Space-Saving (b) HeavyKeeper (c) Double-Anonymous Sketch

Fig. 2: Structure of related works.

Heap-like Solutions utilize heap properties to dynamically manage a list of candidate items. A classic example is **Space-Saving** [37] (see Fig. 2a), which employs a data structure known as Stream-Summary to track frequently occurring items. It maintains frequency counters for selected items within this summary. When a new item arrives, if its ID is not already present, it can replace an item with the smallest frequency. While SS effectively provides a compact representation of item frequencies, it has precision issues due to its failure to fully account for the characteristics of the data stream. **Unbiased Space-Saving** (USS) [42] enhances SS by improving the unbiasedness of frequency estimation. However, its effectiveness is limited in multi-node scenarios, which is beyond the scope of this paper.

CMS-like solutions use multiple hash functions to alleviate the influence of hash collisions and provide reliable frequency estimation. This kind of structure allows for scalable performance in scenarios where data volume is high and rapid updates are necessary. A notable example is **HeavyKeeper** [38] (see Fig. 2b), which utilizes a compact hash table to store hot items while implementing a count-with-exponential-decay strategy to gradually kick out the cold items. However, the decay process can lead to frequency underestimations. This reduces the precision of identifying Top-k items in the tail with similar frequencies, resulting in inaccurate rankings or even exclusions from the Top-k report.

Bucket-connected solutions partition items into buckets to accelerate updates and queries. A well-known example is **Double-Anonymous Sketch** [40] (see Fig. 2c), which is designed to ensure fairness in global Top-k detection. DAS also takes into account the skewness of item distribution, which is prevalent in most data streams, thereby improving accuracy. However, similar to USS, DAS is better suited for scenarios with multiple nodes. Additionally, its inherent two-layer data structure increases processing time, which results in reduced throughput.

Beyond these Top-k sketches, Cuckoo Heavy Keeper (CHK) [43] targets threshold-based heavy-hitter detection. It adopts a fixed lobby-heavy organization, where exponential decay filters items in the lobby before potential heavy hitters are promoted to fingerprint-based heavy entries. Its cuckoo mechanism primarily resolves collisions among candidates within the fixed heavy part. In contrast, HeavyMatcher targets precise Top-k retention and dynamically evolves each bucket by merging compact FP entries and converting potential Top-k items into full-ID entries. Its cuckoo relocation is further coupled with bucket evolution to balance ID-entry occupancy across buckets.

To sum up, existing Top-k sketches still have room for optimization in terms of accuracy and space efficiency; however, optimizing for one inevitably compromises the other. This motivates our work, which aims to achieve perfect Top-k detection while ensuring extremely compact space usage.

C. Insights About Motivation

As previously discussed, the optimal linear space for perfect Top-k detection is only k entries. However, all entry-based Top-k sketches are limited by hash collisions, preventing them from reaching this optimum. To illustrate the gap, we take a simulated dataset that follows a Zipf distribution with a skewness of 1.5 as an example, varying k values to enhance our understanding of the motivations behind this work. This dataset was chosen for its generality, as it reflects the high skewness characteristics of real-world data stream, allowing us to infer how existing algorithms are likely to perform across a range of practical datasets. In particular, we measure the minimum number of entries required by these algorithms to achieve perfect Top-k detection.

For this highly skewed workload, the findings in Fig. 3 indicate that these Top-k sketch algorithms demand one to two orders of magnitude more space than the optimum, especially at larger k. Note that the required space of CMS decreases slightly from Top-700 to Top-800. For each k, we gradually increase the memory budget and report the first evaluated configuration at which CMS achieves 100% Top-k precision. Since the evaluated memory configurations are discrete, different configurations produce different hash-collision patterns, while different k values change which items near the Top-k boundary must be distinguished. As these boundary items have relatively close frequencies in the Zipf-1.5 workload, collision-induced estimation errors may shift the first successful configuration, resulting in the observed non-monotonic threshold.

This confirms that none can simultaneously achieve compact space requirements and perfect Top-k detection. More importantly, despite decades of advancements in Top-k sketches, the trade-off between accuracy and space efficiency remains a significant challenge. This persistent issue fuels our passion and motivates us to develop an extraordinary sketch that moves towards optimal performance in both aspects. In the following

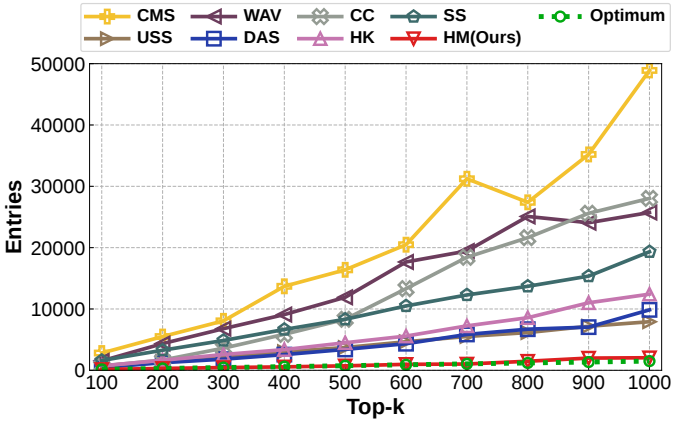


Fig. 3: Space requirement of various Top-k algorithms.

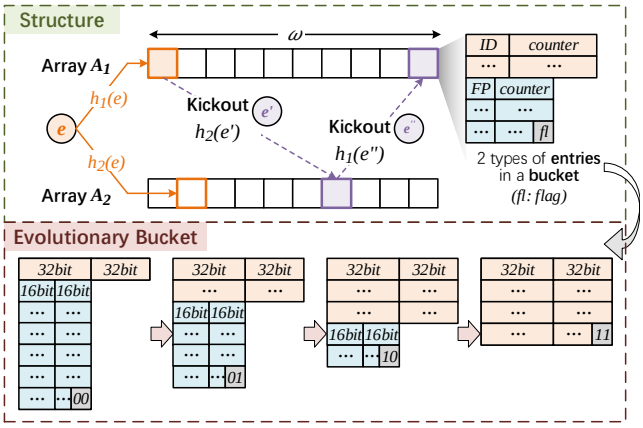


Fig. 4: The data structure of HeavyMatcher.

sections, we will elaborate on our design and present the techniques that contribute to achieving this remarkable goal.

III. The Design of HeavyMatcher

In this section, to achieve compact linear space for perfect Top-k detection, we first examine the characteristics of practical datasets. This observation forms the basis for the design rationale behind HeavyMatcher. We then present the specifics of the data structure and elaborate on the algorithm and its operations. Finally, a running example is provided for better understanding.

A. Observation and Rationale

Although the trade-off between accuracy and space efficiency for Top-k tasks is difficult to overcome in most scenarios, our work focuses on the scenarios of highly skewed data streams. This emphasis is grounded in our observations from real-world datasets that exhibit significant skewness. Specifically, 95% of items in CAIDA, Big CAIDA, Campus, and IMC have frequencies below only 0.33%, 0.05%, 0.01%, and 0.3% of the hottest item's frequency in the corresponding dataset, respectively. Complete cumulative distribution functions are provided in §I of the technical report.

Such imbalance offers an opportunity to break the conventional trade-off, forming the rationale for HeavyMatcher. Specifically, HeavyMatcher propose utilizing a limited number of long ID entries to lock onto hot items while employing a large number of short FP entries to handle the influx of numerous cold items. Built upon this, by integrating entry merging and dynamic evolution techniques, as well as cuckoo hashing for global load balancing, HeavyMatcher effectively protects a greater number of hot items. This approach unifies data structures and algorithms to maximize the matching of candidate Top-k items with memory space. Consequently, HeavyMatcher achieves compact linear space for perfect Top-k detection. It attains **linear space growth** (§IV) and, on the evaluated high-skew workloads, **a near-optimal growth ratio approaching 1** (§V-C4), while consistently outperforming existing sketches in precision, error, and throughput (§V-C2–V-C3).

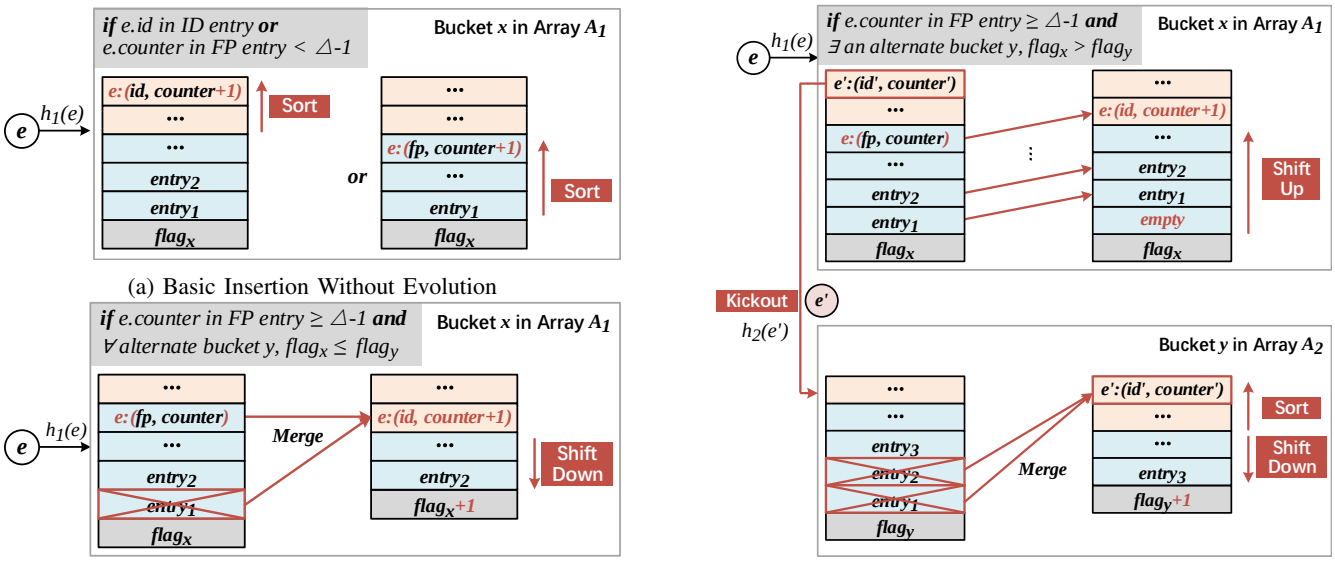
B. Data Structure

As shown in the upper part of Fig. 4, HeavyMatcher consists of two arrays, A_1 and A_2, each comprising ω buckets. A bucket contains a variable number of entries, represented as $\{entry_1, entry_2, \ldots\}$. Here, an entry is a basic unit, and a bucket is an access unit.

To ensure access efficiency, entries are designed to align with multiples of the machine word size. There are two types of entries: ID entries and FP entries, with the length of the former typically being a multiple of the latter. An ID entry includes an *ID* that uniquely identifies the item and a long $counter$, forming a complete <*item ID, frequency*> pair used for recording hot items. In contrast, an FP entry records cold items and consists of a short *fingerprint*, generated via an independent hash function of the *ID*, along with a short $counter$. Furthermore, cuckoo hashing [44], [45] is employed to identify two candidate buckets for an item e, denoted as $h_1(e)$ and $h_2(e)$.

Particularly, the number of ID entries in a bucket increases and FP entries decrease as a data stream arrives, while the memory allocated for each bucket remains fixed. This variation arises from distinct evolutionary processes within buckets, which will be elaborated in §III-C1. For each bucket, we utilize a *flag* to indicate its evolutionary state. The bit length of *flag* is determined by the lengths of both bucket and ID entry, represented as $b_{flag} = \lfloor log_2(bucket_{len}/ID_{len}) \rfloor$. In addition, HeavyMatcher places the *flag* in the trailing bits of $entry_1$, which results in a slight reduction in the length of its $counter$. However, since $entry_1$ stores the smallest $counter$ in the bucket, this placement has minimal impact.

The lower part of Fig. 4 shows an intuitive example of an evolutionary bucket with a fixed size of 256 bits, where the FP entry length is 32 bits and the ID entry length is 64 bits—twice that of an FP entry. At this point, the bucket can accommodate four evolutionary states, so the *flag* occupies the last 2 bits of $counter$ in $entry_1$. Initially, each bucket contains one ID entry and six FP entries, with $flag = \underline{00}$. Through evolution, the bucket will ultimately be filled with four ID entries recording hot items that could be included in the Top-k report, culminating in $flag = \underline{11}$.



(a) Basic Insertion Without Evolution

(b) Insertion with Evolution (Local)

(c) Insertion with Evolution (Load-Balanced)

Fig. 5: The insert operation.

C. Algorithm and Operations

To facilitate description, we denote $A_i[h][r]$ as entry r of bucket h in array i. All entries are initially set to zero. We next describe the two primary operations in HeavyMatcher: insertion and querying.

1) Insert: Pseudocodes are provided in §II of the technical report. When inserting an item e, we first compute two hash indexes, $h_1(e)$ and $h_2(e)$, to locate its candidate buckets. As shown in Fig. 5a, if e is matched (i.e., e already exists in one of its candidate buckets, assumed to be $A_1[x]$), we increment $e.counter$ by 1. Otherwise, indicating that e is a new item for the structure, we check for empty entries in both candidate buckets. Upon finding an empty entry, we place e and set $e.counter$ to 1. If both candidate buckets are full, we randomly select an $entry_1$ from either bucket for replacement. This replacement involves directly replacing the *fingerprint* if $entry_1$ is an FP entry, or replacing the *ID* with probability $1/counter$ if it is an ID entry. Differing replacement strategies stem from a dual consideration of efficiency and precision—direct replacement of FP entries prioritizes efficiency, while probabilistic replacement of ID entries allows for precise handling, providing chances for latecomer hot items. Other replacement strategies, such as direct decrement 1 and exponential weakening decay, may also be applied; however, in our implementation, the two strategies yield the best performance in HeavyMatcher. Entries within the bucket are subsequently sorted by their $counter$ values, with hotter items positioned nearer to the top.

After insertion, the simplest case for the process is complete if item e is either in an ID entry or if the updated value of $e.counter + 1$ in an FP entry remains below the threshold Δ. This threshold is dynamically updated in two steps. First, during insertion, HeavyMatcher maintains three global statistics: $counter^{ID}_{\max}$, $counter^{ID}_{\min}$, and N_{ID}, which denote the maximum and minimum counter values over active ID entries, and the number of active ID entries, respectively. Following the commonly observed Zipf-like skewness in practical data streams, HeavyMatcher estimates the stream skewness parameter $\alpha = \ln(counter^{ID}_{\max}/counter^{ID}_{\min})/\ln(N_{ID})$ online. Second, according to the Zipf frequency law, where the frequency of the k-th hottest item approximately follows $1/k^{\alpha}$, the evolution threshold is updated as $\Delta = \beta \cdot counter^{ID}_{\max}/k^{\alpha}$, where $\beta \in (0, 1]$ controls the aggressiveness of evolution. A larger α results in a smaller threshold, promoting potential Top-k items earlier, whereas a smaller α increases the threshold to avoid frequent evolution among similarly frequent items.

However, if the updated $e.counter+1$ in an FP entry reaches the threshold Δ, e may be a hot item for Top-k reporting, thereby triggering an evolution process that promotes e to an ID entry. Depending on the status of its alternate buckets, HeavyMatcher handles this with two cases: local evolution and load-balanced evolution, which are detailed below.

Local Evolution: Before evolution, the *flag* values of alternate buckets for all ID entries in current bucket are queried. Assume that the bucket $A_1[x]$, where item e is inserted into an FP entry, has the lowest evolution state $flag_x$ compared to all alternate buckets of ID entries in it. Fig. 5b illustrates a basic evolution that occurs locally when the updated $e.counter + 1$ meets the threshold Δ. At this point, e is regarded as a hot item and the FP entry containing e is upgraded to a corresponding ID entry. This upgrade involves merging the memory of upper FP entries as a single ID entry while converting the <*e.fp, counter*> pair into a precise $e.id$ and an extended $counter$ field. Notably, since evolution occurs only when e is matched, $e.id$ can be directly obtained without additional calculations regarding $e.fp$. In the example depicted in Fig. 4, two 32-bit FP entries are combined to form one 64-bit ID entry. Additionally, other original items in the FP entries are shifted down, and the item at the bottom, considered the least

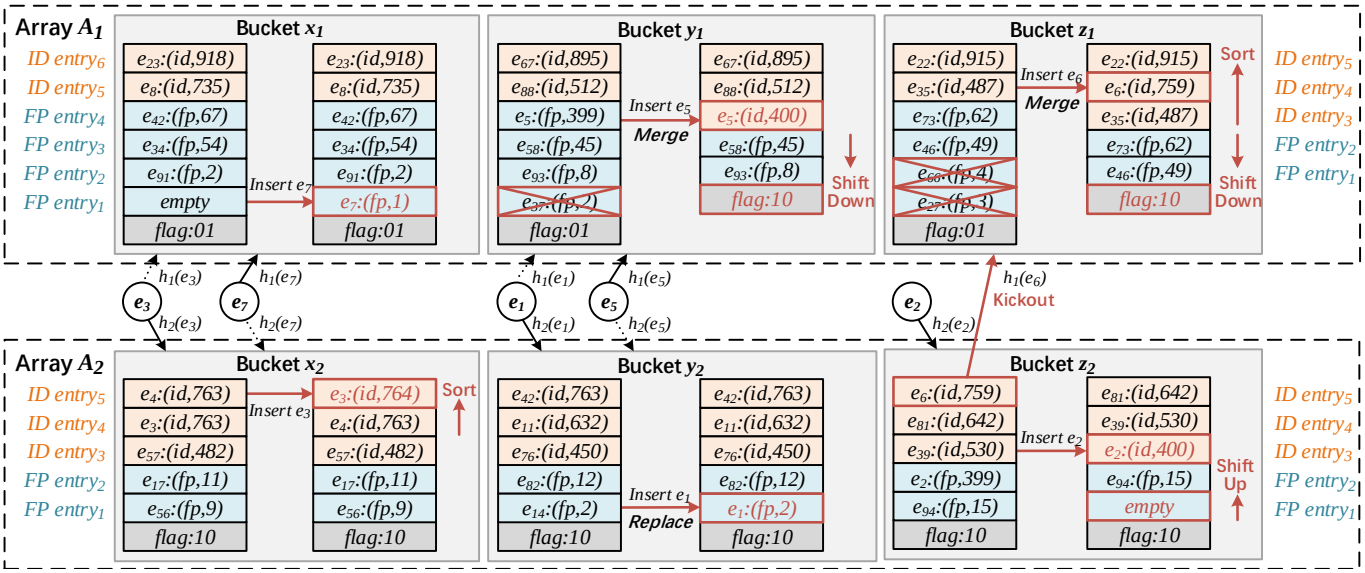


Fig. 6: A running example of HeavyMatcher.

significant in bucket $A_1[x]$, is discarded. Finally, the state $flag_x$ is incremented by 1 to indicate the completion of evolution.

Load-balanced Evolution: Building on local evolution, a load balancing strategy utilizing cuckoo hashing is implemented. This method prevents a bucket from rapidly reaching its final state, characterized by the complete filling of ID entries, such as $flag = \underline{11}$ in the previous example in Fig. 4.

Specifically, as shown in Fig. 5c, for a bucket $A_1[x]$ containing item e in an FP entry to be upgraded, assume there exists an item e' in an ID entry whose alternate bucket $A_2[y]$ ($y = h_2(e')$) has an evolution state $flag_y$ lower than that of $A_1[x]$ (i.e., $flag_x > flag_y$). In this case, before upgrading e, item e' is kicked out to that bucket $A_2[y]$, prompting other items in the ID entries to shift upward and create a bottom ID entry for recording e. Item e then occupies the vacated space for its upgrade with the updated *<e.id, counter+1>* pair, followed by the upward shifting of the remaining items in the FP entries. Additionally, in the alternate bucket $A_2[y]$, items in the FP entries are shifted down, with the bottom two discarded to create space for the kicked-out item e'. e' is subsequently inserted into an appropriate ID entry through sorting. The above process can occur iteratively across alternate buckets until the kicking process ends or the maximum number of kick-outs is reached, ultimately resulting in an increase of 1 in *flag* of an alternate bucket. In our practice, due to the rarity of this case being triggered, it is considered to have almost no impact on the overall efficiency of the insertion process.

2) Top-k Query: In the context of Top-k queries, our focus is exclusively on the items recorded in ID entries. The process begins by extracting all ID entries from both arrays, with each entry represented as a pair $< e.id, counter >$. These pairs are then organized using a sorting algorithm that arranges them in descending order based on their *counter* values, thereby prioritizing items with the highest estimated frequency. After sorting, we select the Top-k items corresponding to the k largest *counter* values to generate a final report as $\hat{\Phi} = \{< \hat{e}_i.id, counter_i >\}_{1 \le i \le k}$.

D. A Running Example

In this section, we present a running example using the 256-bit bucket shown in Fig. 4 to demonstrate the insertion operation, as depicted in Fig. 6. We assume a threshold of $\Delta = 400$. Only the relevant buckets are included.

Case 1: Inserting e_3. After hashing, a search within the two candidate buckets indicates that the item e_3 exists in ID $entry_4$ of bucket x_2. Hence, the *counter* is incremented by 1, and the entry is updated to $e_3 : (id, 764)$. Next, the ID entries in the bucket are sorted, causing e_3 to shift up to $entry_5$, while e_4 moves down to $entry_4$.

Case 2: Inserting e_7. After searching, we find that the item e_7 is not present in its candidate buckets; however, there is an empty FP $entry_1$ in its candidate bucket x_1. As a result, we insert e_7 into this entry as $e_7 : (fp, 1)$.

Case 3: Inserting e_1. The item e_1 is also not found in its candidate buckets, and both are full. Therefore, an FP $entry_1$ of bucket y_2 is randomly selected to insert e_1 by directly replacing the *fingerprint*, resulting in FP $entry_1$ becoming $e_1 : (fp, 2)$.

Case 4: Inserting e_5. Upon searching, we locate the item e_5 in the FP $entry_4$ of bucket y_1. The *counter* for e_5 is thus incremented by 1, reaching 400 and hitting the threshold. After examining the *flag* values of the alternate buckets for e_{67} and e_{88} in ID entries, we find that bucket y_1 has the lowest *flag* value of $\underline{01}$. At this point, bucket y_1 will evolute locally for e_5 to be upgraded. First, e_{37} in the bottom FP $entry_1$ is discarded. Then, e_{93} and e_{58} from the original FP $entry_2$ and $entry_3$ shift down to the new $entry_1$ and $entry_2$, respectively. Next, the vacated space in the original FP $entry_3$ merges with FP $entry_4$, updating it to the new ID $entry_3$ as $e_5 : (id, 400)$. Finally, the *flag* value of bucket y_1 is incremented by 1 to $\underline{10}$.

Case 5: Inserting e_2. To insert item e_2, found in FP $entry_2$ within bucket z_2, we first increment its *counter* by 1, thus

reaching the threshold that triggers evolution. Subsequently, upon examining the alternate buckets for e_6, e_{81}, and e_{39} in the ID entries, we observe that the *flag* value of e_6's alternate bucket, z_1, is $\underline{01}$, the smallest and lower than the *flag* value of the current bucket z_2, which is $\underline{10}$. So, the ID $entry_5$ in bucket z_2, originally containing e_6, is vacated to accommodate the newly upgraded e_2 as $e_2 : (id, 400)$, which is then positioned in ID $entry_3$. Next, the remaining e_{94} in the original FP $entry_1$ shifts up, leaving the bottom of bucket z_2 empty. Additionally, e_6 is kicked out to its alternate bucket z_1. In this process, we check the alternate buckets of e_{22} and e_{35} in the ID entries and find that their *flag* values are not less than that of bucket z_1. Consequently, e_{27} and e_{66} in the bottom FP entries of bucket z_1 are discarded to facilitate the placement of e_6. Following this, the remaining e_{46} in the original FP $entry_3$ and e_{73} in $entry_4$ are shifted down, allowing the vacated FP entries to merge into a new ID $entry_3$. Then, e_6 is moved to the appropriate ID $entry_4$ through sorting. Finally, the *flag* of bucket z_1 increases by 1 to $\underline{10}$.

IV. Mathematical Analysis

We present only the theoretical conclusions below; detailed proofs and simulations appear in §III of the technical report.

Theorem 1. *For the overestimation error of Top-k item e_i ($1 \le i \le k$), there exists η ($\eta > 1$) such that*

$$E(X_i) \le \frac{N - f_i}{(\omega 2^{b+1} - 1) f_i + N} \left((1 - P_i) \cdot \frac{f_i}{\eta^\alpha} + P_i \cdot \frac{\sqrt{\eta^\alpha}}{(\sqrt{\eta^\alpha} - 1)^2} \right), \tag{1}$$

where $P_i = \left(1 - \frac{1}{2\omega}\right)^{max(\eta i - k, 0)}$ and b denotes the fingerprint bit length.

Theorem 2. *For the underestimation error of Top-k item e_i ($1 \le i \le k$), there exists η ($\eta > 1$) such that*

$$E(Y_i) \le {\lambda_1}^2 N \left(\frac{1}{2\omega} - \frac{B}{n} \right) \left((1 - P_i) \cdot \frac{f_i}{\eta^\alpha} + P_i \cdot \frac{\sqrt{\eta^\alpha}}{(\sqrt{\eta^\alpha} - 1)^2} \right). \tag{2}$$

Theorem 3. *To perfectly detect the Top-k items, there exists η ($\eta > 1$) such that the minimum expected number of required entries satisfies*

$$E(entry) = \Theta(k). \tag{3}$$

Theorem 3 shows that the minimum number of required entries grows linearly with k. §V-C4 provides supporting details and experiments, where theory and practice align closely.

Theorem 4. *Let c denote the maximum number of cuckoo kick-outs permitted for an update,. The worst-case per-update time complexity is $O(c+1)$. Consequently, when c is configured as a fixed constant, the per-update time complexity is $O(1)$.*

V. Performance Evaluation

A. Experiment Setup

The experiments were performed on a server with an AMD R9 7940H, 32 cores, and 128GB DRAM. We evaluated Top-k sketch algorithms on the following datasets.

CAIDA: Anonymous IP traces collected by the Equinix Chicago monitor in 2018 [46]. (1) The standard CAIDA dataset consists of 0.11M items with 2.49M insertions, where most items have a frequency below 57, and the maximum item frequency is 17.05K. (2) The Big CAIDA dataset is an expanded version that contains 27.55M insertions and 0.54M items, most of which have a frequency below 80, while the maximum item frequency is 0.17M. In the experiments, Big CAIDA is abbreviated as "B-CD".

Campus: Traces of diverse user activities and network behaviors collected from campus gateways. This dataset captures diverse user activities and network behaviors within a real campus environment, as described in [2]. It consists of ten IP tracking traces collected from campus gateways, encompassing a total of 10M insertions and 0.26M items. Most item frequencies are below 46, with a maximum item frequency of 0.42M.

IMC: A large-scale dataset capturing production-scale traffic patterns from data centers [47]. It consists of 6.71M items that account for a total of 19.86M insertions, with a maximum item frequency of 5.95M. Most items have a frequency of less than 0.3% of this maximum.

Zipf: Synthetic datasets generated with skewness ranging from 1.0 to 1.5 [48]. The frequencies of the Top-k items exhibit a predictable downward trend. Each Zipf dataset contains 32M item insertions. As the skewness increases, the number of items decreases: when the skewness is 1.0, there are 0.97M items, and when the skewness is 1.5, there are 0.12M items. The maximum item frequency ranges from 2.22M to 12.26M.

We report representative results on CAIDA and Zipf-1.5; corresponding results on Campus, IMC, B-CD, and Zipf-1.0 are provided §IV-A of the technical report. §IV-B of the technical report evaluates uniform and mildly skewed workloads under otherwise identical settings.

B. Comparing Solutions and Metrics

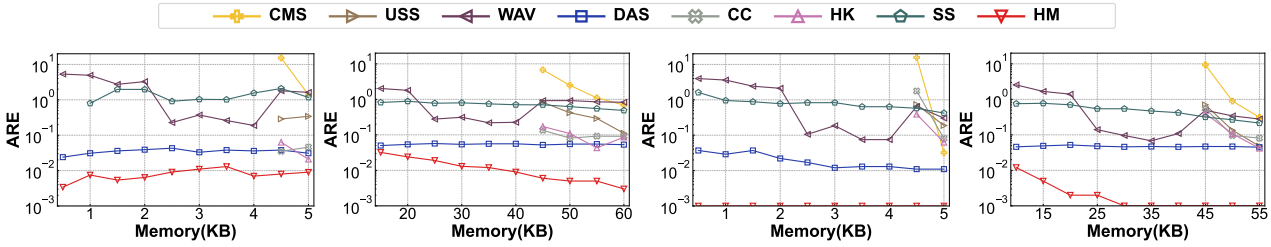
We compare HeavyMatcher against seven representative sketches: Count-Min Sketch (CMS) [39], Unbiased Space-Saving (USS) [42], Waving Sketch (WAV) [49], Double-Anonymous Sketch (DAS) [40], Cuckoo Counter (CC) [15], HeavyKeeper (HK) [38], and Space-Saving (SS) [37]. Note that for sketches requiring a minimum feasible memory (e.g., CMS, SS), evaluation starts from that minimum configuration. Evaluation is conducted on the following metrics.

Error: (1) Average Relative Error (ARE): $\frac{1}{|\Omega|} \sum_{i \in |\Omega|} \left| \frac{f_i - \hat{f}_i}{f_i} \right|$, where f_i and $\hat{f}_i$ are the true and estimated frequencies of item e_i, and Ω denotes the set of items correctly reported as Top-k. (2) Average Absolute Error (AAE): $\frac{1}{|\Omega|} \sum_{i \in |\Omega|} \left| f_i - \hat{f}_i \right|$.

Precision (PRE): Fraction of correctly reported Top-k items, obtained by comparing each algorithm's output with the true Top-k set.

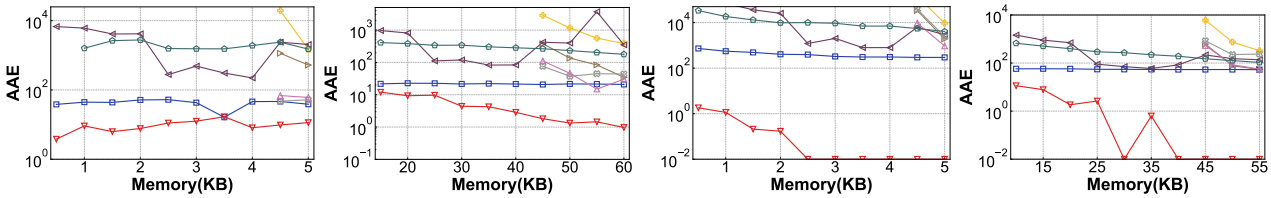
Throughput (THP): Insertion speed measured as N/T, where N is the number of insertions and T is the running time, expressed in million insertions per second (Mips).

Space Requirement: The minimum number of entries required to reach 100% precision, with every 64 bits counted as one entry for clarity.



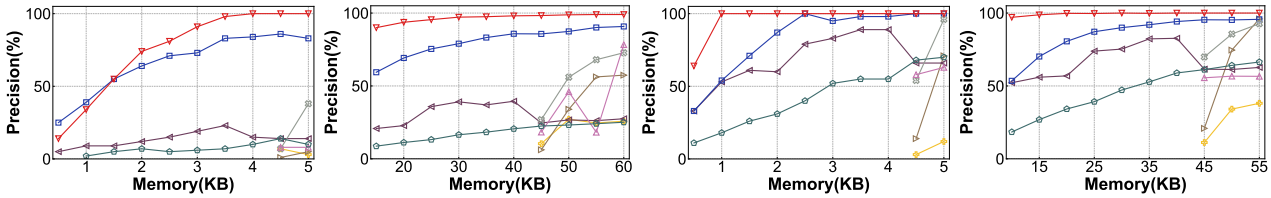
(a) CAIDA, Top-100 (b) CAIDA, Top-1000 (c) Zipf-1.5, Top-100 (d) Zipf-1.5, Top-1000

Fig. 7: Representative ARE comparison under different datasets and Top-k tasks.



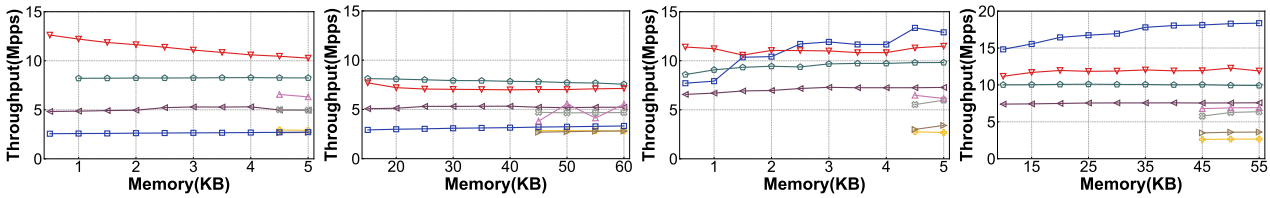
(a) CAIDA, Top-100 (b) CAIDA, Top-1000 (c) Zipf-1.5, Top-100 (d) Zipf-1.5, Top-1000

Fig. 8: Representative AAE comparison under different datasets and Top-k tasks.



(a) CAIDA, Top-100 (b) CAIDA, Top-1000 (c) Zipf-1.5, Top-100 (d) Zipf-1.5, Top-1000

Fig. 9: Representative precision comparison under different datasets and Top-k tasks.



(a) CAIDA, Top-100 (b) CAIDA, Top-1000 (c) Zipf-1.5, Top-100 (d) Zipf-1.5, Top-1000

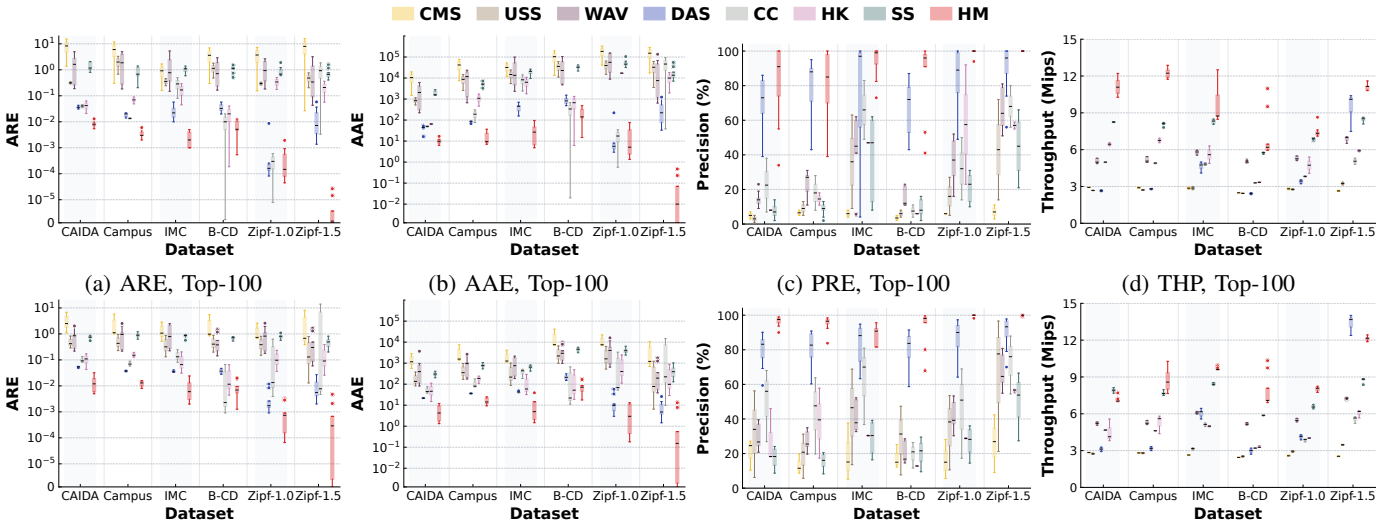
Fig. 10: Representative throughput comparison under different datasets and Top-k tasks.

C. Experiment Results

1) Error: Figs. 7 and 8 present representative ARE and AAE results. HeavyMatcher consistently achieves ARE that is orders of magnitude lower than competing sketches. Specifically, compared with DAS, ARE is reduced by about one order of magnitude; against other algorithms, it is reduced by as much as four orders of magnitude. AAE follows the same trend, with HeavyMatcher also yielding the smallest errors. On the highly skewed Zipf-1.5 dataset, it even reduces errors to zero with modest memory. Note that when an algorithm's precision drops to zero, its ARE and AAE are excluded from the plots.

2) Precision: Fig. 9 presents representative precision results for all algorithms. Most sketches achieve very low precision and are nearly unusable under the same memory. However, HeavyMatcher demonstrates a clear advantage for the Top-1000 task across all datasets, with 100% precision reached more quickly and using less memory. In the Top-100 task, DAS performs well at small memory but fail to converge as memory grows on CAIDA; on Zipf-1.5, although DAS and HeavyMatcher ultimately converge to the same precision, HeavyMatcher converges faster with less memory.

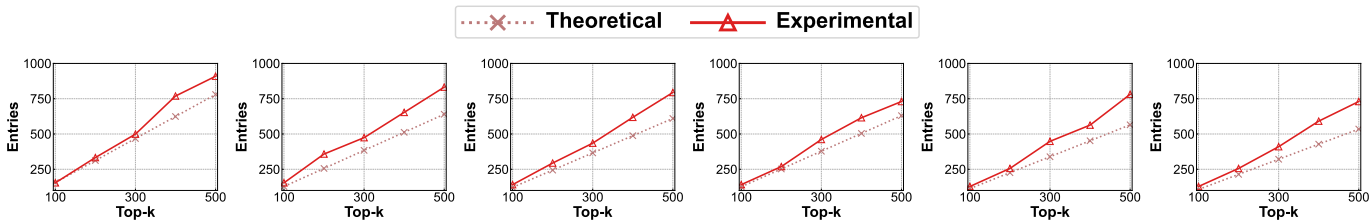
3) Throughput: Fig. 10 presents representative throughput results for all sketch algorithms. HeavyMatcher shows a slight decline as memory increases, but the trend stabilizes once precision converges. While DAS achieves high throughput with the Zipf-1.5 dataset, it proves inefficient with other real-world datasets. HeavyMatcher's strong throughput stems from bounded operations on small, fixed-size buckets: matching, merging, and sorting touch only a few contiguous entries and thus provide strong cache locality. Cuckoo kick-outs and the associated cross-bucket accesses occur only during load-balanced evolution rather than on every update, limiting their average overhead. Under the evaluated compact-memory configurations, bucket accesses can effectively benefit from the cache hierarchy. Notably, much larger bucket arrays may incur



(a) ARE, Top-100 (b) AAE, Top-100 (c) PRE, Top-100 (d) THP, Top-100

(e) ARE, Top-1000 (f) AAE, Top-1000 (g) PRE, Top-1000 (h) THP, Top-1000

Fig. 11: Statistical comparison across datasets and Top-k tasks.



(a) Zipf-1.0 (b) Zipf-1.1 (c) Zipf-1.2 (d) Zipf-1.3 (e) Zipf-1.4 (f) Zipf-1.5

Fig. 12: Evaluation of minimum required entry numbers for perfect Top-k detection.

more cache misses and lower absolute throughput.

In summary, HeavyMatcher delivers high precision with markedly lower memory consumption and a substantial error advantage across Top-k tasks. It consistently outperforms competitors in precision across memory levels, while its throughput remains competitive with the fastest algorithms. These trends are further supported by Fig. 11: across the 12 dataset-task combinations, HeavyMatcher achieves the lowest median ARE and AAE in all 12 cases and the highest median precision in 11 cases, while its median throughput ranks among the top two in all 12 cases. Furthermore, the following experiments investigate HeavyMatcher's memory performance when pursuing 100% precision.

4) Space Requirement: In this experiment, Zipf datasets with varying degrees of skewness and different Top-k tasks are used to evaluate space requirement performance. The comparison between HeavyMatcher and other algorithms at a skewness of $\alpha = 1.5$ is shown in Fig. 3, highlighting its significant advantage. Similar overwhelming benefits were observed in our experiments under other skewnesses, which are omitted due to page limitations. Thus, we illustrate the growth ratio of HeavyMatcher across varying skewness compared to the theoretical simulation results in §IV, as shown in Fig. 12. The results indicate that HeavyMatcher exhibits near-optimal linear space relative to k, with a growth ratio averaging 1.43 and remaining below 1.66. In contrast, as previously analyzed, existing Top-k sketches consume significantly more memory, with growth ratios averaging one to two orders of magnitude higher than the optimum.

D. Practical Application

1) Background: Top-k traffic monitoring is critical to network incident response [7], where operators identify high-volume flows associated with DDoS flooding, scanning, or abuse and often trigger rate limiting or blocking automatically [50]. Inaccurate detection is especially harmful under persistent mitigation: false negatives repeatedly allow malicious traffic to evade suppression, whereas false positives continuously throttle legitimate users. Reliable Top-k identification is therefore essential for effective mitigation with low collateral damage.

2) System Workflow: Fig. 13 illustrates the monitor-react workflow of persistent network incident response. A continuous traffic stream is processed in monitoring windows. In window t, the incident-affected traffic is first observed through traffic monitoring. Measurement primitives such as Space-Saving and HeavyMatcher then identify the Top-k suspicious flows. The detected flows are inserted into the mitigation list, where each entry stores the flow ID and remaining mitigation lifetime (*TTL*). These targets are then subject to traffic control actions, such as rate limiting or traffic blocking. As mitigation persists for T windows, decisions in the current window di-

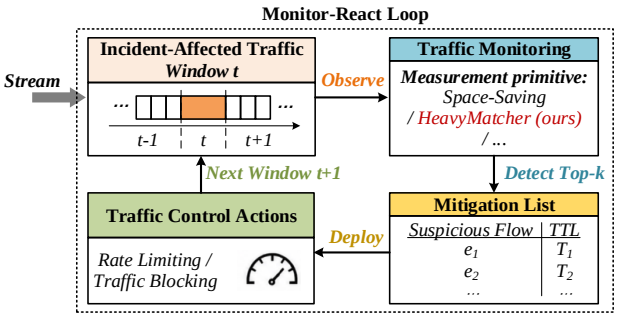


Fig. 13: Workflow of persistent network incident response.

rectly influence traffic behavior in the next monitoring window $(t+1)$, naturally forming a persistent monitor-react loop.

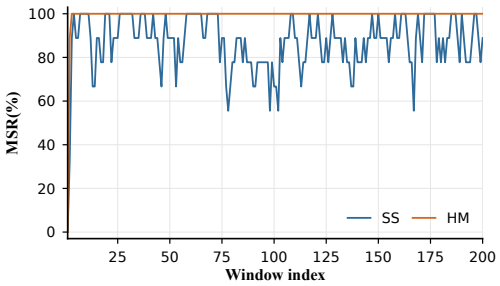
3) Experiment Setup: We evaluate the effectiveness of HeavyMatcher in the persistent mitigation setting described above using the Big CAIDA trace [46] as background traffic. To emulate realistic network incidents, we inject malicious attack flows into the trace. Since many attacks (e.g., DDoS flooding and scanning) exhibit persistent, high-frequency behaviors [51], [52], the injected attack flows are configured to persist across monitoring windows with consistently high packet rates. We set the mitigation duration to $T = 10$ windows. We compare HeavyMatcher with the representative Top-k baseline, Space-Saving, under an identical memory budget of 0.5KB for both methods. Since the objective is reliable persistent mitigation rather than one-shot detection accuracy, we focus on the following system-level metrics:

- **Mitigation Success Rate (MSR):** The fraction of malicious traffic successfully mitigated under persistent mitigation, defined as $\mathrm{MSR} = \frac{\sum_{w\in\mathcal{W}} P_{\mathrm{mit}}(w)}{\sum_{w\in\mathcal{W}} P_{\mathrm{mal}}(w)}$, where $P_{\mathrm{mal}}(w)$ denotes the number of malicious packets in monitoring window w, and $P_{\mathrm{mit}}(w)$ denotes the malicious packets successfully mitigated during their mitigation lifetime.
- **Benign Damage Rate (BDR):** The fraction of benign traffic unnecessarily affected by mitigation actions, defined as $\mathrm{BDR} = \frac{\sum_{w\in\mathcal{W}} P_{\mathrm{dam}}(w)}{\sum_{w\in\mathcal{W}} P_{\mathrm{ben}}(w)}$, where $P_{\mathrm{ben}}(w)$ denotes the number of benign packets in monitoring window w, and $P_{\mathrm{dam}}(w)$ denotes the benign packets unnecessarily affected by mitigation actions.

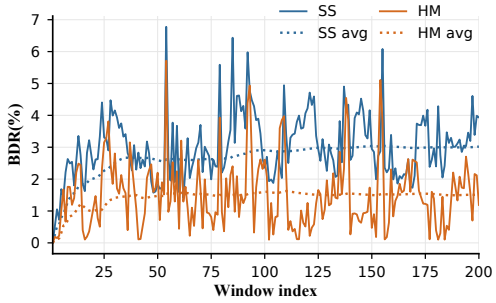
4) Experiment Results: Fig. 14a shows that HeavyMatcher maintains nearly 100% MSR, whereas Space-Saving fluctuates and sometimes falls below 60%; thus, HeavyMatcher more reliably retains malicious Top-k flows for stable suppression. Fig. 14b shows that HeavyMatcher reduces average BDR by approximately 50%, limiting unnecessary mitigation of benign traffic while maintaining strong suppression. Overall, HeavyMatcher provides a more reliable basis for persistent incident response. Its deployment scope and hardware data-plane feasibility are discussed in §V of the technical report.

VI. Conclusion

In this paper, we introduced HeavyMatcher, a novel sketch algorithm designed for Top-k detection with perfect precision and compact linear space usage. By integrating entry merging, evolution, and load balancing techniques, HeavyMatcher effectively locks onto hot items while addressing the trade-off between precision and memory efficiency. Our theoretical analysis establishes a linear relationship between the minimum required number of entries and k for perfect Top-k detection in skewed datasets. Experimental results on high-skew workloads validate this growth ratio, with an average of 1.43 and a maximum of 1.66, indicating that HeavyMatcher is a near-optimal solution. Moreover, HeavyMatcher consistently outperforms existing Top-k sketches in precision while reducing estimation error by up to four orders of magnitude. Finally, a practical deployment in a persistent network incident response system demonstrates that HeavyMatcher improves mitigation effectiveness while reducing benign traffic damage by nearly 50%, highlighting its practical value for real-world Top-k monitoring and mitigation applications.



(a) Mitigation Success Rate



(b) Benign Damage Rate

Fig. 14: Persistent mitigation performance.

VII. Acknowledgment

We thank the anonymous reviewers and our shepherd for their valuable comments and suggestions. This work was supported in part by the Major Key Project of Peng Cheng Laboratory (PCL2025A07), the Young Top-notch Talent Project of Guangdong Province (2023TQ07X362), the Key Research and Development Project of Jiangsu Province (BE2023004-4), the NSFC/RGC Collaborative Research Scheme (62461160332 & CRS_HKUST602/24), the Basic Research Enhancement Program of China (2021-JCJQ-JJ-0483), the Shenzhen Science and Technology Program (ZDCY20250901101035012), the 2025 Research and Development of Security Detection and Defense Software for AI Models project from China Unicom Intelligent Manufacturing Technology & Industry (Guangdong) Co., Ltd. (YGS25AE1000001), and the Shenzhen Postdoctoral Research Fund (DZ31000010).

REFERENCES

[1] Q. Huang, X. Jin, P. P. Lee, R. Li, L. Tang, Y.-C. Chen, and G. Zhang, "Sketchvisor: Robust network measurement for software packet processing," in *ACM SIGCOMM*, 2017.

[2] T. Yang, Y. Zhou, H. Jin, S. Chen, and X. Li, "Pyramid sketch: A sketch framework for frequency estimation of data streams," in *VLDB*, 2017.

[3] Q. Huang, P. P. C. Lee, and Y. Bao, "Sketchlearn: relieving user burdens in approximate measurement with automated statistical inference," in *ACM SIGCOMM*, 2018.

[4] L. Tang, Q. Huang, and P. P. Lee, "SpreadSketch: Toward invertible and network-wide detection of superspreaders," in *IEEE INFOCOM*, 2020.

[5] T. Yang, S. Gao, Z. Sun, Y. Wang, Y. Shen, and X. Li, "Diamond sketch: Accurate per-flow measurement for big streaming data," *IEEE Transactions on Parallel and Distributed Systems*, vol. 30, no. 12, pp. 2650–2662, 2019.

[6] T. Yang, J. Jiang, P. Liu, Q. Huang, J. Gong, Y. Zhou, R. Miao, X. Li, and S. Uhlig, "Elastic sketch: Adaptive and fast network-wide measurements," in *ACM SIGCOMM*, 2018.

[7] M. Yu, L. Jose, and R. Miao, "Software defined traffic measurement with opensketch," in *USENIX NSDI*, 2013.

[8] K. Mouratidis, K. Li, and B. Tang, "Marrying top-k with skyline queries: Operators with relaxed preference input and controllable output size," *ACM Transactions on Database Systems*, vol. 50, no. 1, pp. 1–37, 2025.

[9] Y. Song, B. Yao, Z. Chen, X. Yang, J. Xie, F. Li, and M. Chen, "Efficient top-k spatial-range-constrained approximate nearest neighbor search on geo-tagged high-dimensional vectors," *The VLDB Journal*, vol. 34, no. 1, pp. 1–21, 2025.

[10] Z. Luo, L. Li, M. Zhang, W. Hua, Y. Xu, and X. Zhou, "Diversified top-k route planning in road network," in *VLDB*, 2022.

[11] Y. Dong, C. Xiao, H. Chen, J. X. Yu, K. Takeoka, M. Oyamada, and H. Kitagawa, "Continuous top-k spatial–keyword search on dynamic objects," *The VLDB Journal*, vol. 30, no. 2, pp. 141–161, 2021.

[12] Z. Shu, H. Feng, T. Taleb, and Z. Zhang, "A novel combinatorial multi-armed bandit game to identify online the changing top-k flows in software-defined networks," *Computer Networks*, vol. 230, p. 109783, 2023.

[13] J. Chen, Y. Chen, S. Cai, S. Yin, L. Zhao, and Z. Zhang, "An optimized feature extraction algorithm for abnormal network traffic detection," *Future Generation Computer Systems*, vol. 149, pp. 330–342, 2023.

[14] V. Bruschi, S. Pontarelli, J. Tollet, D. Barach, and G. Bianchi, "Flowfight: High performance–low memory top-k spreader detection," *Computer Networks*, vol. 196, p. 108239, 2021.

[15] Q. Shi, Y. Xu, J. Qi, W. Li, T. Yang, Y. Xu, and Y. Wang, "Cuckoo counter: Adaptive structure of counters for accurate frequency and top-k estimation," *IEEE/ACM Transactions on Networking*, vol. 31, no. 4, pp. 1854–1869, 2023.

[16] L. Tang, Q. Huang, and P. P. Lee, "Mv-sketch: A fast and compact invertible sketch for heavy flow detection in network data streams," in *IEEE INFOCOM*, 2019.

[17] Z. Liang, Y. Li, Z. Dong, H. Wang, W. Li, Q. Huang, Z. Li, and W. Zhang, "Celestialeye: A synergistic cpu-fpga engine for accurate frequency measurement in high-speed data streams," in *ACM/IEEE DAC*, 2026.

[18] Z. Liang, S. Lan, Y. Li, Z. Li, W. Li *et al.*, "FlowTurbo: From best-effort to hit-driven megaflow hardware offloading in open vswitch," in *ACM SIGCOMM*, 2026.

[19] M. Li, D. Chen, H. Dai, R. Xie, S. Luo, R. Gu, T. Yang, and G. Chen, "Seesaw counting filter: A dynamic filtering framework for vulnerable negative keys," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 12, pp. 12 987–13 001, 2023.

[20] M. Li, R. Xie, D. Chen, H. Dai, R. Gu, H. Huang, W. Dou, and G. Chen, "A pareto optimal bloom filter family with hash adaptivity," *The VLDB Journal*, vol. 32, no. 3, pp. 525–548, 2023.

[21] W. Li and P. Patras, "Stable-sketch: A versatile sketch for accurate, fast, web-scale data stream processing," in *ACM WWW*, 2024.

[22] C. Estan and G. Varghese, "New directions in traffic measurement and accounting," in *ACM SIGCOMM*, 2002.

[23] H. Huang, J. Yu, Y. Du, J. Liu, H. Dai, and Y.-E. Sun, "Memory-efficient and flexible detection of heavy hitters in high-speed networks," in *ACM SIGMOD*, 2024.

[24] H. Wang, H. Dai, S. Chen, M. Li, R. Gu *et al.*, "Bamboo filters: Make resizing smooth and adaptive," *IEEE/ACM Transactions on Networking*, 2024.

[25] H. Dai, M. Shahzad, A. X. Liu, and Y. Zhong, "Finding persistent items in data streams," in *VLDB*, 2016.

[26] Q. Shi, C. Jia, W. Li, Z. Liu, T. Yang, J. Ji, G. Xie, W. Zhang, and M. Yu, "Bitmatcher: Bit-level counter adjustment for sketches," in *IEEE ICDE*, 2024.

[27] L. Cao, Q. Shi, W. Xiao, N. Wang, W. Li, Z. Li, W. Zhang, and M. Xu, "Hypersistent sketch: Enhanced persistence estimation via fast item separation," in *IEEE ICDE*, 2025.

[28] Q. Shi, X. Li, H. Zheng, T. Yang, Y. Wang, and M. Xu, "Heavylocker: Lock heavy hitters in distributed data streams," in *ACM SIGKDD*, 2025.

[29] J. Wang, Q. Shi, X. Liang, H. Wang, W. Li, Z. Wei, W. Zhang, and S. Chen, "Pssketch: Finding persistent and sparse flow with high accuracy and efficiency," in *ACM SIGKDD*, 2025.

[30] Y. Li, J. Huang, J. Liu, Z. Li, W. Jiang, and J. Wang, "Switchtop-k: Scaling top-k compression on programmable switches," in *ACM SIGKDD*, 2025.

[31] M. Charikar, K. Chen, and M. Farach-Colton, "Finding frequent items in data streams," *Theoretical Computer Science*, vol. 312, no. 1, pp. 3–15, 2004.

[32] M. Chen, S. Chen, and Z. Cai, "Counter tree: A scalable counter architecture for per-flow traffic measurement," *IEEE/ACM Transactions on Networking*, vol. 25, no. 2, pp. 1249–1262, 2016.

[33] H. Dai, M. Li, A. X. Liu, J. Zheng, and G. Chen, "Finding persistent items in distributed datasets," *IEEE/ACM Transactions on Networking*, vol. 28, no. 1, pp. 1–14, 2019.

[34] T. Li, S. Chen, and Y. Ling, "Per-flow traffic measurement through randomized counter sharing," *IEEE/ACM Transactions on Networking*, vol. 20, no. 5, pp. 1622–1634, 2012.

[35] Y. Li, R. Miao, C. Kim, and M. Yu, "Flowradar: A better netflow for data centers," in *USENIX NSDI*, 2016.

[36] R. Schweller, Z. Li, Y. Chen, Y. Gao, A. Gupta, Y. Zhang, P. A. Dinda, M.-Y. Kao, and G. Memik, "Reversible sketches: enabling monitoring and analysis over high-speed data streams," *IEEE/ACM Transactions on Networking*, vol. 15, no. 5, pp. 1059–1072, 2007.

[37] A. Metwally, D. Agrawal, and A. E. Abbadi, "Efficient computation of frequent and top-k elements in data streams," in *Springer ICDT*, 2005.

[38] T. Yang, H. Zhang, J. Li, J. Gong, S. Uhlig, S. Chen, and X. Li, "HeavyKeeper: An accurate algorithm for finding top-k elephant flows," *IEEE/ACM Transactions on Networking*, vol. 27, no. 5, pp. 1845–1858, 2019.

[39] G. Cormode and S. Muthukrishnan, "An improved data stream summary: the count-min sketch and its applications," *Journal of Algorithms*, vol. 55, no. 1, pp. 58–75, 2005.

[40] Y. Zhao, W. Han, Z. Zhong, Y. Zhang, T. Yang, and B. Cui, "Double-anonymous sketch: Achieving top-k-fairness for finding global top-k frequent items," in *ACM SIGMOD*, 2023.

[41] R. Sanders, "The pareto principle: its use and abuse," *Journal of Services Marketing*, vol. 1, no. 2, pp. 37–40, 1987.

[42] D. Ting, "Data sketches for disaggregated subset sum and frequent item estimation," in *ACM SIGMOD*, 2018.

[43] V. Q. Ngo and M. Papatriantafilou, "Cuckoo heavy keeper and the balancing act of maintaining heavy hitters in stream processing," in *VLDB*, 2025.

[44] R. Pagh and F. F. Rodler, "Cuckoo hashing," *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.

[45] B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher, "Cuckoo filter: Practically better than bloom," in *ACM CoNEXT*, 2014.

[46] "The CAIDA anonymized internet traces," http://www.caida.org/data/overview, 2026.

[47] "Data set for imc 2010 data center measurement," https://pages.cs.wisc.edu/~tbenson/IMC10_Data.html, 2010.

[48] A. Rousskov and D. Wessels, "High-performance benchmarking with web polygraph," *Software: Practice and Experience*, vol. 34, no. 2, pp. 187–211, 2004.

[49] J. Li, Z. Li, Y. Xu, S. Jiang, T. Yang, B. Cui, Y. Dai, and G. Zhang, "Wavingsketch: An unbiased and generic sketch for finding top-k items in data streams," in *ACM SIGKDD*, 2020.

[50] J. Mirkovic and P. Reiher, "A taxonomy of ddos attack and ddos defense mechanisms," in *ACM SIGCOMM*, 2004.

[51] C. Rossow, "Amplification hell: Revisiting network protocols for ddos abuse," in *ISOC NDSS*, 2014.

[52] Y. Xiao, X. Luo, Y. Jiang, A. Wang, H. Chen, Z. Zhou, H. Yu, J. Cao, Y. Jiang, J. Wang *et al.*, "Ddos detection at the scale of one hundred tbps," in *USENIX NSDI*, 2026.